ASSIGNMENT - 5.5

2303A51091
Batch : 29

Task Description #1 (Transparency in Algorithm Optimization)
Task: Use AI to generate two solutions for checking prime numbers:
• Naive approach(basic)
• Optimized approach
Prompt:
“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”
Expected Output:
• Code for both methods.
• Transparent explanation of time complexity.
• Comparison highlighting efficiency improvements.

task1.py > ...

```python

# -------- Optimized Method --------
def is_prime_optimized(n):
    if n <= 1:
        return False

    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False

    return True


# -------- Main Program --------
num = int(input("Enter a number: "))

# Naive Result
if is_prime_naive(num):
    print("Naive Method:", num, "is Prime")
else:
    print("Naive Method:", num, "is NOT Prime")

# Optimized Result
if is_prime_optimized(num):
    print("Optimized Method:", num, "is Prime")
else:
    print("Optimized Method:", num, "is NOT Prime")

```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-5.5.py/task1.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-5.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-5.5.py/task1.py"
Enter a number: 5
Naive Method: 5 is Prime
Optimized Method: 5 is Prime
arepallyramcharan@Arepallys-MacBook-Air assignment-5.5.py %
```

Observation:

1. Naive method checks all numbers from 2 to n-1 → Time Complexity O(n).
2. Optimized method checks only up to √n → Time Complexity O(√n).

3. Both methods give the same correct result.
4. Optimized version significantly reduces iterations for large numbers.
5. Performance improvement becomes more noticeable as input size increases.

Task Description #2 (Transparency in Recursive Algorithms)
Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.
Instructions:
1. Ask AI to add clear comments explaining recursion.
2. Ask AI to explain base cases and recursive calls.
Expected Output:
• Well-commented recursive code.
• Clear explanation of how recursion works.
• Verification that explanation matches actual execution.

task2.py > ...

```python
def fibonacci(n):
    # Base Case 1:
    # If n is 0, return 0
    # This stops recursion when we reach the first Fibonacci number
    if n == 0:
        return 0

    # Base Case 2:
    # If n is 1, return 1
    # This stops recursion when we reach the second Fibonacci number
    if n == 1:
        return 1

    # Recursive Case:
    # Fibonacci of n is the sum of Fibonacci of (n-1) and (n-2)
    # The function calls itself until it reaches the base cases
    return fibonacci(n - 1) + fibonacci(n - 2)


# Taking user input
num = int(input("Enter a number: "))

print("Fibonacci number is:", fibonacci(num))
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-5.5.py/task2.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-5.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-5.5.py/task2.py"
Enter a number: 8
Fibonacci number is: 21
arepallyramcharan@Arepallys-MacBook-Air assignment-5.5.py %
```

Observation:

The recursive function clearly demonstrates how a problem is broken into smaller subproblems.

Base cases (`n == 0` and `n == 1`) correctly prevent infinite recursion.

Recursive calls follow the mathematical definition of Fibonacci: $F(n) = F(n-1) + F(n-2)$.

The function produces correct results, but it performs repeated calculations.

Time complexity is **$O(2^n)$**, making it inefficient for large inputs.

Task Description #3 (Transparency in Error Handling)
Task: Use AI to generate a Python program that reads a file and processes data.
Prompt:
“Generate code with proper error handling and clear explanations for each exception.”
Expected Output:
• Code with meaningful exception handling.
• Clear comments explaining each error scenario.
• Validation that explanations align with runtime behavior.

```python
def read_and_process_file(filename):
        # Close the file after reading
        file.close()

    except FileNotFoundError:
        # This error occurs if the file does not exist
        print("Error: The file was not found.")

    except PermissionError:
        # This error occurs if the program does not have permission to read the file
        print("Error: Permission denied to access the file.")

    except Exception as e:
        # This catches any other unexpected errors
        print("Unexpected error occurred:", e)

    else:
        # This block runs only if no exceptions occurred
        print("File processed successfully.")

    finally:
        # This block runs whether an error occurs or not
        print("Execution completed.")


# Taking user input
filename = input("Enter the file name: ")
read_and_process_file(filename)
```

Output:

```
/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-5.5.py/task3.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-5.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-5.5.py/task3.py"
Enter the file name: 4
Error: The file was not found.
Execution completed.
arepallyramcharan@Arepallys-MacBook-Air assignment-5.5.py %
```

Observation:

1. Program prevents crashes using structured exception handling.
2. Each exception type is clearly separated.
3. Explanations match actual runtime behavior.

4. finally ensures execution completes gracefully.
5. Improves reliability and transparency of file operations.

Task Description #4 (Security in User Authentication)
Task: Use an AI tool to generate a Python-based login system.
Analyze: Check whether the AI uses secure password handling practices.
Expected Output:
• Identification of security flaws (plain-text passwords, weak validation).
• Revised version using password hashing and input validation.
• Short note on best practices for secure authentication.

task4.py > ...

```python
import hashlib
import getpass

# Function to hash passwords using SHA-256
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

# Stored credentials (hashed password)
stored_username = "admin"
stored_password_hash = hash_password("SecurePass123")  # Hashed password

# Input from user
username = input("Enter username: ").strip()

# Hide password input while typing
password = getpass.getpass("Enter password: ").strip()

# Input validation
if not username or not password:
    print("Username and password cannot be empty.")
else:
    # Hash entered password
    entered_password_hash = hash_password(password)

    # Compare hashed passwords
    if username == stored_username and entered_password_hash == stored_password_hash:
        print("Login successful")
    else:
        print("Invalid credentials")
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-5.5.py/task4.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-5.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-5.5.py/task4.py"
Enter username: admin
Enter password:
Invalid credentials
arepallyramcharan@Arepallys-MacBook-Air assignment-5.5.py %
```

Observation:

Never store passwords in plain text.

Always hash passwords using strong algorithms (bcrypt recommended over SHA-256).

Use salting to prevent rainbow table attacks.

Implement account lockout after multiple failed attempts.

Use HTTPS in real applications.

Validate inputs properly.

Store credentials in secure databases, not directly in code.

Use multi-factor authentication (MFA) when possible.

Task Description #5 (Privacy in Data Logging)
Task: Use an AI tool to generate a Python script that logs user activity (username, IP address, timestamp).
Analyze: Examine whether sensitive data is logged unnecessarily or insecurely.
Expected Output:
• Identified privacy risks in logging.
• Improved version with minimal, anonymized, or masked logging.
• Explanation of privacy-aware logging principles.

```python
task5.py > ...
import datetime

def log_user_activity(username, ip_address):
    timestamp = datetime.datetime.now()

    with open("activity_log.txt", "a") as file:
        file.write(f"Username: {username}, IP: {ip_address}, Time: {timestamp}\n")

# Example usage
username = input("Enter username: ")
ip = input("Enter IP address: ")

log_user_activity(username, ip)
print("Activity logged successfully.")
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-5.5.py/task5.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-5.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-5.5.py/task5.py"
Enter username: admin
Enter IP address: 12345
Activity logged successfully.
arepallyramcharan@Arepallys-MacBook-Air assignment-5.5.py %
```

Observation:

Data Minimization

- Log only what is necessary.

- Avoid storing sensitive information.

Anonymization / Pseudonymization

- Hash or mask identifiers.

Access Control

- Restrict who can access logs.

Encryption

- Encrypt log files at rest.

Retention Policy

- Delete logs after defined period.

Transparency

- Inform users about logging practices.